
Hybrid Mechanical and Agentic Specification Inference: A Study

Anonymous Author(s)

Affiliation

Address

email

Abstract

Formal verification stalls from the cost of specification: to establish high-level facts, contracts have to be written for many supporting functions. Specification inference is one technique which can help by deriving specs from programs which can then be used to prove the primarily interesting properties. In this paper, we present a study for inference in the Move smart contract language, which has a specification language built in and strong verification support. We use an agent to run inference, and equip it with mechanical tools based on *weakest precondition analysis*. Inference performance and quality is measured for three different inference tactics: pure agent inference, and two forms of hybrid inference with WP, while running against Opus 5, Sol 5.6, and Terra 5.6. As a corpus we use extracts from a real-world private product, avoiding bias from known code examples. The study shows multiple interesting results: first, pure model based spec inference works surprisingly well, tackling hard problems which require the agent to invent lemmas and helpers. Second, different models make different use of WP: the mid-level model, Terra, benefits most by a 25% reduction in API cost, whereas the higher end models do less. All have a lower error rate using WP. In comparison, with help of WP, Terra solves with the same success rate than the other models, but with much lower cost.

1 Introduction

A roadblock in program verification adoption is the *cost of specification*. The properties developers care about are high-level — a call chain aborts only under stated preconditions, an entrypoint satisfies its postconditions, stored data respects an invariant — but to prove any of them, the supporting functions need contracts of their own, and those are boilerplate: they restate what the code does. *Specification inference* derives such contracts from the program, so that effort can go to the properties worth stating by hand. Verifiers abstract from imperative code with Dijkstra’s weakest-precondition transformation (WP) [9], and WP is the natural inference engine, but it stops at loops: a loop needs an invariant as induction hypothesis [6], invariant synthesis has no complete technique despite a long history [8], and in practice invariants are written by hand.

LLMs have been applied to inference since 2023 [20, 15], and with today’s frontier models a well-instructed agent that receives the verifier’s diagnosis can realistically infer specifications, loop invariants included. This paper is a study of such inference for Move, a smart contract language with a specification language built in and strong verification support in the Move Prover (MVP) [10, 13]. We run inference with a coding agent, and equip it with a mechanical WP tool that derives precise, simplified contracts once the loop invariants are in place: the agent asks for specs, the tool either succeeds or reports the loops that lack an invariant, the agent proposes invariants, and the cycle repeats.

The setup is measured on a corpus of 20 inference problems drawn from a private codebase (that of a production perpetual trading system) — private so that contracts must be inferred rather than recalled — with three tactics, one without the WP tool and two with, under three models, Opus 5, Sol 5.6, and Terra 5.6, and with completeness checked by held-out mutants. The study shows several things at once. Pure agent-based inference works surprisingly well: at its first attempt it produced a complete specification in 95% of its cells under the two frontier models, including on targets that require inventing lemmas and recursive helper functions. Different models make different use of the WP tool: relative to the unaided agent it costs about $1.3\times$ under Opus 5, $0.95\times$ under Sol 5.6, and $0.X\times$ under Terra 5.6 — a trade of exact contracts for context, and which side pays depends on how the model spends its tokens. In every case the success rate with WP is higher: the tool removes the incomplete contracts the unaided agent leaves behind, all of them on the targets whose contract has to pin down a whole data structure.

MVP was developed for the Diem project and is since maintained by the blockchain company behind it; it has verified Diem’s payment system and critical components of a public blockchain [19, 3]. Move shares Rust’s safe references, linear types, algebraic data types, and higher-order functions [4]; the mechanical half of our approach relies on exactly these traits — above all reference elimination (Sec. 2.2) — so we expect it to carry over to verifiers for Rust-like languages.

Sec. 2 presents hybrid inference, Sec. 3 the evaluation, and Sec. 4 discusses the results and concludes.

2 Hybrid Inference

Hybrid inference combines two sources of specifications: a mechanical WP analysis, which is exact where it applies and stops at loops, and a coding agent, which proposes what the analysis cannot derive — loop invariants and idiomatic, readable contracts — and repairs against verifier feedback. The verifier is the oracle for both. We first walk through an inference session (Sec. 2.1), then describe the WP analysis (Sec. 2.2) and the tactics and skills that drive the agent (Sec. 2.3).

2.1 Inference Session Example

We illustrate the workflow of specification inference in our system by examples. The examples are written in Move, but should be understandable without knowing details about the language. For a reference, see the Move Book [4].

Consider a coding agent like Claude Code. The Move Prover is connected to the agent via an MCP service. The service offers (a) a tool for launching verification and (b) a tool for running WP-based spec inference. The agent has a set of skills which describe language, verification techniques, and the MCP tools in detail.

The example we look at is an iterative computation of the exponential, b^e :

```

user
fun pow(base: u64, exp: u64): u64 {
  let result = 1;
  let i = 0;
  while (i < exp) {
    result = result * base;
    i = i + 1;
  }
  result
}
```

Based on the instructions, the first thing the agent does is call WP. The answer points to the missing loop invariant, including unrolling the loop symbolically and showing the values of the affected loop variables at each entry:

```

wp tool
WP inferred 'vacuous' conditions after this loop without an invariant. ...
--> sources/pow.move:5:14
5 |         while (i < exp) {
  |           ^
```

```

|
= loop-invariant evidence
= bounded loop-head facts :
  head[0]: head[0].i = 0
           head[0].result = 1
  head[1]: 0 < exp  $\implies$  head[1].i = 1
           0 < exp  $\implies$  head[1].result = base
  head[2]: 1 < exp  $\implies$  head[2].i = 2
           1 < exp  $\implies$  head[2].result = base * base
  ..

```

Based on the diagnosis, the agent next derives the loop invariant. A recursive specification helper function is introduced for accumulating the result. Before returning the result, the agent uses the verification tool to check whether its solution verifies:

```

fun pow(base: u64, exp: u64): u64 {
  ..
  while (i < exp) {
    result = result * base;
    i = i + 1;
  } spec {
    invariant [inferred] i  $\leq$  exp;
    invariant [inferred] result = math_pow(base, i);
  }
  ..
}
// specification helper function
spec fun math_pow(base: num, exp: num): num {
  if (exp  $\leq$  0) { 1 } else { math_pow(base, exp - 1) * base }
}

```

In the next turn the agent runs the WP MCP tool again. With the loop invariant present, the complete specification is derived. However, the aborts condition is marked as `sathard` by the WP tool. The marker is set when a derived condition is exact but its satisfiability is expected to be hard for the solver — here because of the existential quantification, which says that the computation will abort if for some y the exponent multiplication overflows. Note that the Move Prover uses unbounded integers, `num`, in the specification language, making it possible to reason about overflow in the programming language's `u64`:

```

spec pow { // post and aborts condition for pow
  ensures [inferred] result = math_pow(base, exp);
  aborts_if [inferred = sathard]  $\exists$  y: u64:
    math_pow(base, y) * base > MAX_U64
    && ( $\forall$  x: num: 0  $\leq$  x && x  $\leq$  y  $\implies$  math_pow(base, x)  $\leq$  MAX_U64)
    && y < exp
}

```

The agent tries to verify this specification, but it times out with the following diagnosis:

```

verification failed:
error: verification out of resources/timeout (timeout set to 5s)
= timeout analysis replay: mixed quantifier and nonlinear arithmetic
= top quantifier activity:
  quantifier at sources/pow.move:24 -- 55,793+ observed instantiations

```

The quantifier the diagnosis is pointing to is the defining recursive axiom of the specification helper function `math_pow`, which is apparently endlessly unfolded in order to prove the `sathard` aborts condition. In the next turn, the agent introduces lemmas to characterize the monotonicity of multiplication, which allows them to rewrite the aborts condition:

```

spec pow {
  pragma opaque;
  ensures [inferred] result = math_pow(base, exp);
  /// The partial products are non-decreasing, so the loop
  /// overflows exactly when its last multiplication does.
  aborts_if [inferred] exp > 0 && math_pow(base, exp - 1) * base > MAX_U64;
} proof {
  /// Bounds the multiplication of every iteration by the last one; the
  /// trigger fires on the 'math_pow(base, i)' of the loop invariant.
  ∀ k: num {math_pow(base, k)} apply math_pow_last_is_max(base, k, exp);
}
spec lemma math_pow_last_is_max(base: num, k: num, n: num) {
  requires base ≥ 0 && 0 ≤ k && k < n;
  ensures math_pow(base, k) * base ≤ math_pow(base, n - 1) * base;
} proof {
  apply math_pow_mono(base, k + 1, n);
}
spec lemma math_pow_mono ... // left out for brevity

```

94

95 2.2 Weakest-Precondition Analysis

96 We run a textbook WP analysis [9, 6] over the bytecode MVP hands to its Boogie back-end, and read
 97 off abort conditions, modifies clauses, and consequence-style postconditions. The novelty here is not
 98 in the WP algorithm, but in the *shape of the bytecode it operates on*: by the time the analysis runs,
 99 MVP’s standard transformation pipeline has already eliminated all Move references and injected
 100 every available specification as in-line assume/assert statements. The result is a reference-free,
 101 contract-annotated control flow on which backward predicate propagation reduces to the classical
 102 case.

103 **Reference elimination.** Move has Rust-style references: $\&T$ for shared reads and $\&\text{mut } T$ for exclusive
 104 writes, both with a stack-bounded lifetime. Predicate-transformer reasoning over such references
 105 is awkward, because the meaning of $*r$ depends on which root location r was derived from. MVP
 106 avoids this by rewriting bytecode into a reference-free intermediate form [10]. Shared references are
 107 inlined: $\&T$ disappears and reads through it become reads of the underlying value. Mutable references
 108 are replaced by in/out parameters.

109 **Specification instrumentation.** Before WP runs, the bytecode passes through MVP’s regular
 110 specification instrumentation. Each function-level requires, aborts_if, ensures, or modifies
 111 clause, and each loop invariant, is compiled into an assume or assert at the appropriate program
 112 point. Calls are instrumented uniformly via the *behavioral predicates* of [13]: *requires_of<f>*,
 113 *aborts_of<f>*, *ensures_of<f>*, *modifies_of<f>*, and *result_of<f>* reify a callee’s contract as
 114 predicates over the call site’s arguments and result. Loops are broken at their headers via the standard
 115 *cut-point* encoding of Barnett and Leino for unstructured programs [6], with the loop invariant
 116 playing the role of cut-point predicate: the back-edge is severed, the invariant is asserted before
 117 and after each iteration, and the loop-modified locations are havoced in between, leaving an acyclic
 118 fragment through which WP propagates as through any straight-line code.

119 At the input to WP, the bytecode therefore looks like a sequence of pure assignments interleaved with
 120 assume and assert over plain values; the surface-level features (references, resources, calls, loops,
 121 user specifications) are gone.

122 2.3 Tactics and Skills

123 The agentic side of the workflow is delivered as a plugin to a general-purpose coding agent: skills
 124 that instruct it, hooks that check generated Move as it is written, and the Move toolchain exposed as
 125 tools it can call. The setup is at [2]; we summarize the design.

126 **Inference tactics.** Inference runs in one of three *tactics*, which differ in whether the WP tool is
 127 available and who decides when to use it.

128 – *agent-only*: WP is unavailable. The contract and any loop invariants are derived directly from the
 129 implementation and the contracts of its dependencies, checked as the work proceeds.

130 – *hybrid-guided*: the workflow of Sec. 2.1, prescribed as a fixed order — run WP over the scope,
131 take one warned function and write an invariant from the bounded loop-head facts, rerun WP on
132 that function alone until the warning is gone, simplify what WP derived, then check.

133 – *hybrid-flexible*: WP is available as one inference pass among others, and the agent decides whether
134 and when to call it and how to combine it with its own reasoning.

135 **Skills.** A tactic selects the *plan* — the sequence of tasks and which of them consult WP— and
136 nothing else. Everything a tactic would otherwise have to rediscover is shared reference material,
137 identical in all three tactics: the specification language, the rules for writing and editing specs, the
138 verification sub-workflow, and the toolchain’s capabilities and limits. Only the description of the WP
139 tool itself is withheld where the tool is absent; the underlying idea of backward reasoning, and what a
140 loop without an invariant costs, is background every tactic receives.

141 Three areas of that guidance carry most of the weight.

142 – *Loop invariants*: when to introduce recursive helper functions for iterative quantities, and when
143 to prefer state-level invariants on resources — assumed by the prover at every call site, including
144 each loop iteration — over per-loop inductive arguments.

145 – *Simplification*: discard vacuous conditions; replace unbounded quantifiers, a frequent source of
146 SMT timeouts, with bounded or closed-form equivalents; and require trigger annotations on the
147 surviving ones.

148 – *Timeout resolution*: a hierarchy of escalations — strengthen state-level invariants, then introduce
149 spec helpers, then lemmas, then explicit proof scripts — with a strict prohibition on weakening
150 abort or postconditions to silence a timeout, since this trades a real property for the appearance of
151 progress.

152 Independently of the tactic, every emitted condition is tagged with an [inferred] marker, so that
153 re-runs are idempotent and never disturb user-written specifications.

154 3 Evaluation

155 The evaluation compares along two dimensions. *Arm against arm* (Sec. 3.2) holds the model fixed and
156 varies the tactic: *agent-only*, *hybrid-guided*, and *hybrid-flexible*, run under the same conditions. *Model*
157 *against model* (Sec. 3.3) repeats the design with two further models and asks how the contribution
158 of WP changes with the model. All rounds share the corpus, the scoring, and the cost accounting
159 described first.

160 3.1 Setup

161 **Corpus.** The corpus is drawn from the private Move code of a production perpetual trading system:
162 23 targets from it, two from a public Move library that carry no upstream specification, and two
163 written for the corpus. A private codebase matters here: the model cannot have seen these functions,
164 or any specification of them, during training, so a contract has to be inferred rather than recalled.
165 Every candidate is classified by the features (strata) its contract has to account for — loops and
166 accumulators, early returns and breaks, arithmetic overflow and underflow, casts and truncation,
167 mutable references and in-place permutation, function values, quantifiers, composition through a
168 callee’s contract, and others, 38 in all. A target is admitted only if it proves within seconds against a
169 hand-written reference specification, and before the round three *mutants* are authored per target —
170 patches to the implementation, one per obligation the contract has to pin — each shown to be killed
171 by the reference. Twenty targets are selected to cover all 38 features across 15 modules, 10 of them
172 with loops; three are deliberately guessable as controls, and five near-duplicates are held back. With
173 3 arms and 4 replicates, a model round has **240 cells**.

174 **Arms and models.** The three tactics of Sec. 2.3 run as arms that differ only in the rendered plugin: the
175 reference material is byte-identical, and in agent-only the WP tool is absent rather than discouraged.
176 Three rounds use three models: Opus 5 at the highest effort setting through the Claude Code SDK,
177 and Sol 5.6 and Terra 5.6 at high effort through the Codex CLI, with the same plugin rendered for
178 each. TODO(wg): The Terra 5.6 round is pending; its rows and table currently hold GLM 5.3 data (maximum effort, Claude Code SDK

179 against an Anthropic-compatible endpoint) as a placeholder, marked [†]. Each session has a budget of 100–180k output
180 tokens, one hour of wall time, and 40 seconds of solver time per verification condition.

181 **Scoring.** A cell reaches *operational success* when a judge outside the agent’s workspace confirms that
182 the package compiles, the requested scope verifies, no weakening construct is present — `pragma`
183 `verify = false`, partial abort contracts, vacuous postconditions, invented preconditions — the
184 bytecode is unchanged, and the required contract categories are present. The agent’s own claim is
185 never a score. In some rounds the controller also applied a visible set of mutants after the agent’s
186 first attempt and, if one survived, handed the contract back for strengthening; whether it did so
187 varied between rounds, and the agent was never told a second attempt might follow. Every round
188 is therefore read the same way: a cell’s outcome is the judge’s verdict at the end of the agent’s *first*
189 *attempt*, and its cost is what the agent spent up to then; later strengthening turns are not counted
190 anywhere, and a contract that survived a visible mutant at the first attempt is a *weak contract*, a failure.
191 Operational success is necessary but weak: a vaguer contract passes it more easily. Completeness
192 — postconditions as strong as the behavior — has no deductive check in general; WP guarantees it
193 only given complete loop invariants and callee contracts, and warns when it cannot, a mechanism the
194 agent-only arm lacks. The second level is therefore *mutation adequacy*: after the round, each held-out
195 mutant is applied to the implementation and the prover rerun against the inferred specification. A
196 mutant that still verifies has survived; a cell is *complete* when its first attempt succeeded and killed
197 every held-out mutant. Held-out scoring has finished for the Opus 5 round and is in progress for the
198 other two.

199 **Cost.** Costs are in US dollars at each model’s published prices at the time of its round, recomputed
200 from the recorded token counters rather than taken from the harness’s estimate. Per million tokens,
201 Opus 5 is \$5 uncached input, \$0.50 cache read, \$10 or \$6.25 cache write, and \$25 output; Sol 5.6 is
202 \$4 uncached input, \$0.40 cached input, \$5 cache write, and \$20 output; Terra 5.6 is \$2, \$0.20, \$2.50,
203 and \$12. Thinking counts as output for all three.

204 3.2 Arm against Arm

205 Table 1 summarizes all rounds; Tables 2–4 in the appendix give every target. Under Opus 5, the
206 hybrids succeeded at the first attempt in all 80 cells each and every one of those killed every held-out
207 mutant. Agent-only failed 4 of its 80: all four replicates of QP-part-025, the in-place permutation,
208 produced at the first attempt a contract that pinned where the partition point lands but not what the
209 vector contains, and a visible mutant survived it. That is the completeness effect the mechanical
210 baseline is meant to buy, and it lands on the one target whose contract has to characterize a whole
211 data structure.

212 Cost goes the other way. In the task-weighted mean, hybrid-guided costs $1.29\times$ agent-only and
213 hybrid-flexible $1.27\times$; 95% percentile bootstrap intervals, holding the 20 tasks fixed and resam-
214 pling four replicate blocks within each task with the three arms kept together (100,000 draws),
215 are [1.22, 1.37] and [1.20, 1.35]. The hybrids are cheaper on only 5 and 1 of the 20 targets —
216 TS-trial-019 under hybrid-guided at 54% is the largest saving — and cost most on the function-
217 value target SM-select-022 ($2.0\times$ and $1.5\times$) and the lemma target BA-base-012 ($1.5\times$ and $1.3\times$).
218 The QP-part-025 comparison is not a cost comparison: agent-only’s cheap cells there are failed first
219 attempts.

220 The mechanism is visible in the token and tool counts. The hybrids process two thirds more input
221 (546k and 502k against 328k per cell) and emit a fifth more output (10.6k and 10.8k against 9.0k):
222 the WP output and its simplification enter the context and are re-read on every turn, and at cache-read
223 prices that is where the extra cost lies. They also make more verify calls (160 and 174 against 105)
224 on top of their 150 and 108 WP calls, and more edits (182 and 177 against 131).

225 3.3 Model against Model

226 Under Sol 5.6 the picture shifts. The hybrids again succeeded at the first attempt in all 80 cells each,
227 and agent-only failed 3 with a weak contract. At a near-identical price list, Sol 5.6 spends less per
228 cell than Opus 5 in every arm (\$0.52 against \$0.69 for agent-only), because it emits two thirds of the
229 output tokens; and here WP pays: hybrid-guided and hybrid-flexible cost $0.95\times$ and $0.94\times$ agent-only,
230 [0.85, 1.05] and [0.84, 1.04], and are cheaper on 13 and 15 of the 20 targets. The saving is spread
231 rather than concentrated — a third to a half off on the small arithmetic contracts (VS-shares-002,

Table 1: All rounds, per arm: mean cost in US dollars per cell, the hybrid/agent-only cost ratio with its 95% bootstrap interval, cells that failed at the first attempt, cells that killed every held-out mutant over cells scored (*: scoring in progress; —: not yet scored), mean input/output tokens in thousands, and Edit, verify, and WP calls summed over the arm’s cells, all counted up to the end of the first attempt. [†]Placeholder: GLM 5.3 data until the Terra 5.6 round completes.

model	arm	cost	vs. agent [95% CI]	failed	complete	I/O k	edits	verifies	WP
Opus 5	agent-only	\$0.695	—	4	76/76	328/9.0	131	105	0
	hybrid-guided	\$0.896	1.29 [1.22, 1.37]	0	80/80	546/10.6	182	160	150
	hybrid-flexible	\$0.883	1.27 [1.20, 1.35]	0	80/80	502/10.8	177	174	108
Sol 5.6	agent-only	\$0.517	—	3	19/19*	479/6.6	133	140	0
	hybrid-guided	\$0.491	0.95 [0.85, 1.05]	0	18/18*	450/6.9	160	124	172
	hybrid-flexible	\$0.483	0.94 [0.84, 1.04]	0	18/18*	444/5.8	118	124	145
Terra 5.6 [†]	agent-only	\$0.274	—	2	—	592/17.0	279	157	0
	hybrid-guided	\$0.291	1.06 [0.91, 1.31]	1	—	609/19.4	257	155	144
	hybrid-flexible	\$0.277	1.01 [0.87, 1.18]	2	—	585/18.0	240	161	121

VS-contrib-003, PM-curve-027, BK-bucket-016) — and the two targets that cost the hybrids most under Opus 5 cost them most here too: QP-part-025 at $1.5\times$ and $1.2\times$, BA-base-012 at $1.4\times$ under hybrid-guided. The hybrids’ input volume is no longer higher than agent-only’s (450k and 444k against 479k), because the unaided model reads more of the package instead.

Terra 5.6, the mid-tier model of the same family at a third of Sol’s price list, is expected to lean on WP most: hybrid-guided costs $0.X\times$ agent-only $[0.X, 0.X]$ and hybrid-flexible $0.X\times [0.X, 0.X]$, cheaper on X and X of the 20 targets, and agent-only fails X cells at the first attempt against none for the hybrids. **TODO(wg):** Terra 5.6 round pending; the rows marked [†] in Table 1 and Table 4 hold GLM 5.3 data as a placeholder. Fill in the 0.X figures, then refresh the Sol 5.6 scoring.

Since each model is priced at its own schedule, the comparison across models is between the within-model ratios, not the dollars. Read together, the rounds separate two contributions of WP. The first is completeness, and it does not depend on the model: in every round the success rate with WP is higher. Agent-only left a weak contract at the first attempt in 7 of 160 cells under Opus 5 and Sol 5.6 and X under Terra 5.6, the hybrids in none, and the failures sit on the targets whose contract has to pin down a data structure. The second is cost, and there different models make different use of the tool: WP is not a fixed saving but a trade of exact contracts for context, and a model that writes the invariant unaided and re-reads a large context pays for it (Opus 5, $1.3\times$), a model that reads more and writes less recovers it (Sol 5.6, $0.95\times$), and a smaller model that would otherwise search recovers more (Terra 5.6, $0.X\times$). The Sol 5.6 intervals include parity, so its size is indicative; the spread across the three models is the finding.

4 Discussion and Conclusion

4.1 Interpreting the Results

Sec. 3 shows three things at once: an agent with no mechanical help produced a complete specification at its first attempt in 95% of its cells under the two frontier models; the WP tool removed the remaining 5%, which sit on the targets whose contract has to pin down a data structure; and different models make different use of it, from a $1.3\times$ cost under Opus 5 to $0.X\times$ under Terra 5.6. The question is therefore not whether WP is worth it, but for which model, and on which targets.

Agent-only does surprisingly well. Given no WP tool at all, the direct tactic found the same recursive spec helpers, lemmas, and loop invariants as the hybrids, under Opus 5 and Sol 5.6 alike, and it got there by a recognizable method: it wrote its own mutation probes and let counterexamples drive the repair. Where it fell short, it fell short in one way: a contract that pins a scalar — the partition point, the returned index — and leaves the data structure around it unconstrained, which a mutant that permutes the contents exposes. Every one of the seven weak contracts is of that kind, and the hybrids, which start from the exact postcondition WP derives, left none.

Two things temper that. The targets are individual functions of a few dozen lines, and while several need genuine invention — an invented lemma, a permutation contract, a bound that holds only through

a callee’s contract — none requires reasoning across a module or a call graph of any depth. And the hybrids’ cost is highest on exactly the invention-heavy targets, where the mechanically derived contract is large and the model re-reads it on every turn. We expect both effects to grow with the size of the target, and we have no data on that yet.

Three models, three cost answers. Sol 5.6 is priced like Opus 5 but spends less per session, because it writes fewer tokens and reads more of the package instead; there the hybrids save about 5%, spread over most targets. Terra 5.6, the smaller model of the same family, benefits most, at $0.X\times$. The Sol 5.6 interval includes parity, so we read the sizes as indicative and the spread across the models as the finding. Most of what WP supplies is a diagnosis — *this loop has no invariant, and here are its first iterations* — plus an exact contract that costs context to carry; a model that finds the invariant unaided pays for the context, a model that reads more and writes less recovers it. The completeness effect does not depend on the model, and we read that as the durable part: the class of failure the mechanical component removes — the vacuous postcondition after a havocked loop, the contract that pins a scalar and forgets the structure — is not one that fluency prevents.

Open questions. The evaluation leaves the questions of scale open. Its targets are functions, not modules; its corpus is one codebase in one domain; and its cost measure is dollars under three models’ price schedules. We plan to apply inference retrospectively to a public blockchain framework, which already carries hand-written specifications, and to compare the inferred contracts against them — a comparison this round cannot make, since its targets deliberately have no specification of their own.

4.2 Related Work

Specification inference predates large language models — Daikon [11] learned likely invariants from program traces — but the techniques that followed stayed tied to particular property classes and source languages, and the advent of LLMs has revived interest in the area. Pei et al. [20] showed that fine-tuned LLMs can predict program invariants statically at quality competitive with dynamic analysis. Kamath et al. [15] pair an LLM that *proposes* loop invariants with a symbolic checker that *validates* them, retrying on failure — a propose-and-validate loop that recurs in much of the subsequent work. For Verus, AlphaVerus [1] bootstraps verified code by self-improving translation with tree search over verifier feedback, and VeriStruct [21] synthesizes Verus-style specifications and proofs for Rust functions, modules, and data structures through planner-orchestrated phases. For smart contracts in Solidity, PropertyGPT [17] retrieves human-written Certora properties as input examples for specification generation. Most directly related to our work, MSG [12] targets the same Move specification language through a skill-based agentic design that consumes verifier feedback beyond a binary success/failure verdict.

Across these systems the LLM remains responsible for the entire specification; the verifier serves as a binary oracle or, in MSG and our setup, as a feedback channel, but never as a contributor of mechanically derived content.

A separate line of work, exemplified by DafnyPro [5] and AutoVerus [22], tackles the complementary problem of synthesizing the auxiliary proof annotations a verifier needs when the specification is already given. Our plugin can also synthesize proof hints (Sec. 2.1), but this is not the focus of this paper.

Reference elimination, the bytecode transformation underlying our WP analysis (Sec. 2.2), was introduced in [10], and equivalents exist for Rust. Aeneas [14] compiles Rust to a pure functional model in a target prover (Lean, F*, Coq, HOL4) via forward and backward functions, while RustHorn [18] encodes a mutable borrow as a pair of its current value and a *prophecy* for its value at the end of the borrow, reducing verification to constrained Horn clauses. Neither infers specifications — in Aeneas the user writes them in the target prover, against the translated model — and both consume the reference-free form only in the forward direction. Our use of the transformation is the opposite: WP runs on the reference-free form, but the conditions it derives have to be carried *back* through the elimination, so that what the user reads and edits is stated over the original references rather than over in/out parameters or prophecy variables. To our knowledge this inversion has not been described before; it is what makes a reference-eliminating pipeline usable for inference rather than only for proving. It is straightforward for *path-based* eliminations such as ours and Aeneas’s, where an eliminated reference still corresponds to a location the source can name. Whether it carries to a prophecy encoding, where the borrow’s final value has no source-level counterpart, we do not know.

Our backend IVL, Boogie [7, 16], also performs WP on the procedural code passed in [6] and passes the result on to the SMT solver. One major difference from our approach is that we aim to generate user-readable conditions, in terms of Move, whereas Boogie’s WP is not intended for external consumption. This is reflected in our mapping of references back to the source language, and in the significant effort put into expression simplification, since raw WP predicates can be very verbose.

In summary, two aspects distinguish our setup from the work above. First, to our knowledge, no prior LLM-based spec-inference system uses a sound mechanical baseline alongside the LLM. WP itself produces a substantial part of the inferred specification — abort conditions, modifies clauses, and consequence-style postconditions — so the AI is left only with what WP cannot recover (loop invariants, high-level conservation laws), and the two streams are validated together by the Move Prover. The AI side runs a propose-observe-refine loop like the systems above; the mechanical baseline narrows the LLM’s search space.

Second, the workflow is delivered as a plugin to a general-purpose, interactive coding agent (Claude Code) via skills, edit-time hooks, and an MCP server, rather than as a custom-built tool: the agent participates in the loop and – most importantly – the user can intervene at any point. To our knowledge, no prior LLM-based inference system for verification languages has this kind of coding-agent integration, including agentic ones such as MSG.

4.3 Conclusion

We have presented a specification inference tool for the Move Prover that combines a sound, mechanical WP analysis over Move bytecode with an agentic coding CLI; the prover serves as the oracle for both signals. The two inference streams compose in a single instrumentation pipeline, and the workflow is delivered as a Claude Code plugin — skills, hooks, and an MCP server — so inference runs inside the same conversational loop the user already writes Move in. A controlled comparison on 20 targets drawn from a private perpetual-trading codebase — three tactics, four replicates each, three models — found that a pure agent produces a complete specification at its first attempt in 95% of its cells under the frontier models, even where lemmas and helper functions have to be invented; that with WP the success rate is higher in every round, the tool removing the rest, all on targets whose contract has to pin down a data structure; and that different models make different use of it, from $1.3\times$ the cost under Opus 5 to $0.95\times$ under Sol 5.6 and $0.X\times$ under Terra 5.6. Whether that margin widens with the size of the target is the question we take up next. Neither the WP construction nor the agentic workflow is specific to Move: both rest on reference elimination and on a verifier that consumes contracts, so we expect the approach to transfer to Rust-like languages and their verifiers.

References

- [1] Pranjal Aggarwal, Bryan Parno, and Sean Welleck. AlphaVerus: Bootstrapping formally verified code generation through self-improving translation and tree refinement. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*, volume 267 of *Proceedings of Machine Learning Research*, 2025.
- [2] Anonymous. Specification-inference plugin for the move prover, 2026. Source available on request; released with the camera-ready.
- [3] Aptos Labs. The Aptos Framework Book. <https://aptos-labs.github.io/framework-book/>, 2023. Accessed: 2026-05-09.
- [4] Aptos Labs. The Move on Aptos Book. <https://aptos-labs.github.io/move-book/>, 2023. Accessed: 2026-05-09.
- [5] Debangshu Banerjee, Olivier Bouissou, and Stefan Zetsche. DafnyPro: LLM-assisted automated verification for Dafny programs, 2026. URL <https://arxiv.org/abs/2601.05385>. Dafny Workshop, POPL 2026.
- [6] Mike Barnett and K. Rustan M. Leino. Weakest-precondition of unstructured programs. In *Proceedings of the 6th ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering*, pages 82–87, New York, NY, USA, 2005. Association for Computing Machinery. ISBN 1595932399. doi: 10.1145/1108792.1108813.

- [7] Mike Barnett, Bor-Yuh Evan Chang, Robert DeLine, Bart Jacobs, and K Rustan M Leino. Boogie: A modular reusable verifier for object-oriented programs. In *International Symposium on Formal Methods for Components and Objects*, pages 364–387. Springer, 2005.
- [8] Patrick Cousot and Nicolas Halbwachs. Automatic discovery of linear restraints among variables of a program. In *Proceedings of the 5th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL '78)*, pages 84–96. ACM, 1978. doi: 10.1145/512760.512770.
- [9] Edsger W. Dijkstra. Guarded commands, nondeterminacy and formal derivation of programs. *Communications of the ACM*, 18(8):453–457, 1975. doi: 10.1145/360933.360975.
- [10] David L. Dill, Wolfgang Grieskamp, Junkil Park, Shaz Qadeer, Meng Xu, and Jingyi Emma Zhong. Fast and reliable formal verification of smart contracts with the move prover. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2022)*, volume 13243 of *Lecture Notes in Computer Science*, pages 183–200. Springer, 2022. doi: 10.1007/978-3-030-99524-9_10.
- [11] Michael D. Ernst, Jake Cockrell, William G. Griswold, and David Notkin. Dynamically discovering likely program invariants to support program evolution. *IEEE Transactions on Software Engineering*, 27(2):99–123, 2001. doi: 10.1109/32.908957.
- [12] Yu-Fu Fu, Meng Xu, and Taesoo Kim. Agentic specification generator for Move programs. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE 2025)*, pages 1286–1298, 2025. doi: 10.1109/ASE63991.2025.00110.
- [13] Wolfgang Grieskamp, Teng Zhang, Vineeth Kashyap, and Jake Silverman. Formal verification of imperative first-class functions in Move. *CoRR*, abs/2605.10007, 2026. URL <https://arxiv.org/abs/2605.10007>. Preprint; to appear in FMCAD 2026.
- [14] Son Ho and Jonathan Protzenko. Aeneas: Rust verification by functional translation. *Proc. ACM Program. Lang.*, 6(ICFP):116:1–116:31, 2022. doi: 10.1145/3547647.
- [15] Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop invariants using large language models, 2023. URL <https://arxiv.org/abs/2311.07948>.
- [16] K. Rustan M. Leino and Philipp Rümmer. A polymorphic intermediate verification language: Design and logical encoding. In Javier Esparza and Rupak Majumdar, editors, *TACAS*, pages 312–327, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg. ISBN 978-3-642-12002-2.
- [17] Ye Liu, Yue Xue, Daoyuan Wu, Yuqiang Sun, Yi Li, Miaolei Shi, and Yang Liu. PropertyGPT: LLM-driven formal verification of smart contracts through retrieval-augmented property generation. In *Network and Distributed System Security Symposium (NDSS 2025)*, 2025. doi: 10.14722/ndss.2025.241357.
- [18] Yusuke Matsushita, Takeshi Tsukada, and Naoki Kobayashi. RustHorn: CHC-based verification for Rust programs. *ACM Trans. Program. Lang. Syst.*, 43(4):1–54, 2021. doi: 10.1145/3462205.
- [19] Junkil Park, Teng Zhang, Wolfgang Grieskamp, Meng Xu, Gerardo Di Giacomo, Kundu Chen, Yi Lu, and Robert Chen. Securing Aptos Framework with Formal Verification. In *5th International Workshop on Formal Methods for Blockchains (FMBC 2024)*, volume 118 of *Open Access Series in Informatics (OASIs)*, pages 9:1–9:16. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024. doi: 10.4230/OASIs.FMBC.2024.9.
- [20] Kexin Pei, David Bieber, Kensen Shi, Charles Sutton, and Pengcheng Yin. Can large language models reason about program invariants? In *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, volume 202 of *Proceedings of Machine Learning Research*, pages 27496–27520. PMLR, 2023. URL <https://proceedings.mlr.press/v202/pei23a.html>.
- [21] Chuyue Sun, Yican Sun, Daneshvar Amrollahi, Ethan Zhang, Shuvendu K. Lahiri, Shan Lu, David L. Dill, and Clark Barrett. VeriStruct: AI-assisted automated verification of data-structure modules in Verus. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2026)*, Lecture Notes in Computer Science. Springer, 2026. URL <https://arxiv.org/abs/2510.25015>. arXiv:2510.25015.

[22] Chenyuan Yang, Xuheng Li, Md Rakib Hossain Misu, Jianan Yao, Weidong Cui, Yeyun Gong, Chris Hawblitzel, Shuvendu K. Lahiri, Jacob R. Lorch, Shuai Lu, Fan Yang, Ziqiao Zhou, and Shan Lu. AutoVerus: Automated proof generation for Rust code. *Proc. ACM Program. Lang.*, 9(OOPSLA2), 2025. doi: 10.1145/3763174.

A Per-Target Results

Tables 2–4 give, for each target and arm, the mean cost per cell over the four replicates, mean input/output tokens in thousands (I/O k), the hybrid mean cost as a percentage of agent-only (vs. agent), and whether the specification succeeded at the first attempt and killed every held-out mutant (✓) or failed (✗, subscripted with the count where more than one replicate failed); * marks cells whose scoring is still in progress and ◦ a cell lost to infrastructure. All figures count up to the end of the first attempt; input counts include uncached, cached, and cache-creation tokens.

Table 2: Opus 5 by target. Summed over the 20 targets the arms cost \$55.56, \$71.71, and \$70.65.

target	agent-only			hybrid-guided				hybrid-flexible			
	cost	I/O k	ok	cost	vs. agent	I/O k	ok	cost	vs. agent	I/O k	ok
BA-base-012	\$1.132	552/17.6	✓	\$1.665	147%	1008/25.4	✓	\$1.459	129%	845/21.7	✓
BK-bucket-016	\$0.577	347/6.2	✓	\$0.663	115%	428/6.1	✓	\$0.702	122%	455/7.0	✓
LP-price-021	\$0.612	286/7.4	✓	\$0.653	107%	369/6.8	✓	\$0.589	96%	295/5.7	✓
MD-median-015	\$0.387	155/3.9	✓	\$0.320	83%	162/2.1	✓	\$0.476	123%	231/5.6	✓
MM-min-013	\$0.440	256/4.2	✓	\$0.531	121%	329/5.5	✓	\$0.530	120%	328/5.4	✓
OV-order-006	\$0.353	151/3.7	✓	\$0.305	87%	154/2.0	✓	\$0.405	115%	186/3.9	✓
PM-curve-027	\$0.404	168/4.5	✓	\$0.391	97%	202/3.3	✓	\$0.491	122%	259/4.7	✓
QP-part-025	\$0.934	509/12.2	✗ ₄	\$1.475	158%	826/20.3	✓	\$1.855	199%	1154/25.6	✓
SM-select-022	\$1.160	533/17.9	✓	\$2.298	198%	1493/33.9	✓	\$1.753	151%	927/27.8	✓
TL-lev-020	\$0.649	333/8.0	✓	\$0.995	153%	716/10.3	✓	\$0.832	128%	557/8.6	✓
TR-cancel-026	\$0.855	418/11.0	✓	\$1.310	153%	866/15.4	✓	\$1.075	126%	648/12.2	✓
TR-discard-011	\$0.646	281/8.3	✓	\$0.917	142%	553/10.2	✓	\$0.934	145%	573/10.6	✓
TR-order-010	\$0.548	281/6.2	✓	\$0.605	110%	294/6.6	✓	\$0.607	111%	308/6.5	✓
TS-trial-019	\$0.788	352/12.0	✓	\$0.429	54%	197/4.1	✓	\$0.811	103%	455/10.6	✓
UC-credits-008	\$0.556	234/7.3	✓	\$0.717	129%	414/8.0	✓	\$0.618	111%	300/7.4	✓
VS-contrib-003	\$0.510	216/5.1	✓	\$0.578	113%	343/4.7	✓	\$0.681	134%	399/6.1	✓
VS-fees-001	\$0.877	382/11.3	✓	\$1.253	143%	771/15.8	✓	\$0.891	102%	439/11.1	✓
VS-redeem-004	\$1.394	595/22.4	✓	\$1.710	123%	1178/21.4	✓	\$1.518	109%	851/21.1	✓
VS-shares-002	\$0.668	353/6.5	✓	\$0.566	85%	319/4.4	✓	\$0.821	123%	531/7.9	✓
WU-consume-023	\$0.400	170/4.6	✓	\$0.547	137%	304/5.7	✓	\$0.613	153%	301/7.1	✓
all 20	\$0.695	328/9.0	76/80	\$0.896	129%	546/10.6	80/80	\$0.883	127%	502/10.8	80/80

Table 3: Sol 5.6 by target. Summed over the 20 targets the arms cost \$41.35, \$39.30, and \$38.68.

target	agent-only			hybrid-guided				hybrid-flexible			
	cost	I/O k	ok	cost	vs. agent	I/O k	ok	cost	vs. agent	I/O k	ok
BA-base-012	\$0.657	641/10.0	✓	\$0.938	143%	987/16.2	✓	\$0.617	94%	552/10.8	✓
BK-bucket-016	\$0.574	632/6.3	✓	\$0.373	65%	335/4.8	✓	\$0.464	81%	493/5.0	✓
LP-price-021	\$0.385	330/4.3	✓	\$0.296	77%	190/3.7	✓	\$0.275	71%	167/1.8	✓
MD-median-015	\$0.337	278/3.1	✓	\$0.255	76%	161/2.2	✓	\$0.281	83%	194/1.9	✓
MM-min-013	\$0.384	340/4.4	*	\$0.416	108%	364/5.2	* ₂	\$0.375	98%	352/4.1	* ₂
OV-order-006	\$0.320	276/3.5	✗ ₃	\$0.286	89%	196/2.7	* ₄	\$0.265	83%	187/1.7	* ₄
PM-curve-027	\$0.342	259/3.5	* ₄	\$0.210	61%	123/1.6	* ₄	\$0.257	75%	206/2.3	* ₄
QP-part-025	\$0.989	1091/13.7	* ₄	\$1.518	154%	1710/23.4	* ₄	\$1.172	119%	1210/18.2	* ₄
SM-select-022	\$0.744	653/12.3	* ₄	\$0.815	110%	773/15.2	* ₄	\$0.970	130%	919/16.1	* ₄
TL-lev-020	\$0.517	479/5.5	* ₄	\$0.490	95%	431/6.5	* ₄	\$0.631	122%	643/7.2	* ₄
TR-cancel-026	\$0.547	494/7.3	* ₄	\$0.582	107%	540/10.0	* ₄	\$0.499	91%	489/6.3	* ₄
TR-discard-011	\$0.550	474/6.1	* ₄	\$0.411	75%	372/5.2	* ₄	\$0.517	94%	507/6.1	* ₄
TR-order-010	\$0.446	352/5.3	* ₄	\$0.469	105%	425/5.9	* ₄	\$0.564	126%	461/5.2	* ₄
TS-trial-019	\$0.440	337/6.6	* ₄	\$0.322	73%	207/4.0	* ₄	\$0.268	61%	210/2.2	* ₄
UC-credits-008	\$0.466	424/7.0	* ₄	\$0.601	129%	603/8.6	* ₄	\$0.413	89%	406/4.8	* ₄
VS-contrib-003	\$0.508	532/5.7	* ₄	\$0.269	53%	195/2.5	* ₄	\$0.268	53%	196/1.9	* ₄
VS-fees-001	\$0.527	478/6.7	* ₄	\$0.443	84%	381/6.0	* ₄	\$0.579	110%	566/7.4	* ₄
VS-redeem-004	\$0.816	838/12.3	* ₄	\$0.613	75%	629/8.6	* ₄	\$0.668	82%	743/8.0	* ₄
VS-shares-002	\$0.428	396/4.8	* ₄	\$0.218	51%	141/1.9	* ₄	\$0.291	68%	170/2.1	* ₄
WU-consume-023	\$0.361	285/4.5	* ₄	\$0.301	83%	230/3.8	* ₄	\$0.296	82%	209/3.1	* ₄
all 20	\$0.517	479/6.6	19/80*	\$0.491	95%	450/6.9	18/80*	\$0.483	94%	444/5.8	18/80*

Table 4: Terra 5.6 by target ([†]placeholder: GLM 5.3 data until the Terra 5.6 round completes). Summed over the 20 targets the arms cost \$21.59, \$23.30, and \$22.18.

target	agent-only			hybrid-guided				hybrid-flexible			
	cost	I/O k	ok	cost	vs. agent	I/O k	ok	cost	vs. agent	I/O k	ok
BA-base-012	\$0.237	379/21.1	* ₄	\$0.331	140%	608/27.3	* ₄	\$0.146	62%	265/10.7	* ₄
BK-bucket-016	\$0.133	266/8.0	* ₄	\$0.120	91%	244/7.2	* ₄	\$0.178	134%	372/9.9	* ₄
LP-price-021	\$0.167	338/10.0	* ₄	\$0.177	106%	345/11.3	* ₄	\$0.176	105%	353/10.7	* ₄
MD-median-015	\$0.110	227/6.3	* ₄	\$0.107	97%	238/5.3	* ₄	\$0.105	95%	208/6.1	* ₄
MM-min-013	\$0.120	255/6.5	* ₄	\$0.147	123%	346/6.8	* ₄	\$0.132	111%	302/6.2	* ₄
OV-order-006	\$0.089	159/5.9	* ₄	\$0.080	90%	147/4.9	* ₄	\$0.085	95%	168/4.5	* ₄
PM-curve-027	\$0.131	234/8.2	* ₄	\$0.100	76%	184/6.4	* ₄	\$0.094	72%	170/6.1	* ₄
QP-part-025	\$0.328	702/21.5	○	\$0.977	298%	1887/79.3	* ₄	\$0.656	200%	1534/41.2	* ₄
SM-select-022	\$0.327	665/23.5	* ₄	\$0.343	105%	612/28.8	* ₄	\$0.305	93%	579/23.6	* ₄
TL-lev-020	\$0.276	572/18.4	* ₄	\$0.323	117%	650/21.1	* ₄	\$0.242	87%	540/13.7	* ₄
TR-cancel-026	\$0.385	697/33.4	* ₄	\$0.530	138%	1109/38.1	* ₄	\$0.420	109%	846/27.4	* ₄
TR-discard-011	\$0.229	498/13.6	* ₄	\$0.296	129%	696/15.6	* ₄	\$0.282	123%	582/18.2	* ₄
TR-order-010	\$0.239	419/19.7	* ₄	\$0.297	125%	703/15.8	* ₄	\$0.268	112%	493/21.4	* ₄
TS-trial-019	\$0.217	327/19.2	* ₄	\$0.253	116%	446/20.8	* ₄	\$0.274	126%	443/23.3	* ₄
UC-credits-008	\$0.522	1378/23.7	✗	\$0.431	82%	1144/19.2	✗	\$0.752	144%	1849/42.9	✗ ₂
VS-contrib-003	\$0.738	1984/34.2	✗	\$0.143	19%	296/7.6	* ₄	\$0.202	27%	437/11.5	* ₄
VS-fees-001	\$0.198	366/14.2	* ₄	\$0.319	161%	630/22.8	* ₄	\$0.300	151%	580/22.0	* ₄
VS-redeem-004	\$0.794	1935/39.9	* ₄	\$0.595	75%	1448/32.7	* ₄	\$0.656	83%	1450/42.1	* ₄
VS-shares-002	\$0.123	241/6.8	* ₄	\$0.146	118%	260/9.9	* ₄	\$0.151	123%	305/9.0	* ₄
WU-consume-023	\$0.116	218/7.6	* ₄	\$0.110	95%	189/7.9	* ₄	\$0.121	105%	216/8.6	* ₄
all 20	\$0.274	592/17.0	0/79*	\$0.291	106%	609/19.4	0/80*	\$0.277	101%	585/18.0	0/80*